Intelligent Cache Replacement using ML

Ashutosh Negi
Graphic Era Hill University
Dehradun, Uttarakhand
ashutoshnegi994@gmail.com

Suryansh Chauhan
Graphic Era Hill University
Haridwar, Uttarakhand
suryanshr38@gmail.com

***Abstract*— This paper presents a comprehensive analysis of various classical policies as well as machine learning algorithms for cache replacement in operating system, which provides guidance to the community about machine learning algorithms that can be used in the field of cache replacement, by showing difference between the classical algorithms like LRU (least recently used), MRU (most recently used) and ML algorithms like navies' bayes, decision tree and random forest. The main objective of this research was to decrease the number of cache misses while keeping the cache size the same. We analyzed 39% more cache hits in the best-case scenario than the Classical Algorithms. This states that ML algorithms have an upper hand over the Traditional Algorithms over a pattern-based scenario as far as cache hit ratios are concerned for the operating system level caches as they are highly adaptable over the user's changing usage patterns.**

I. Introduction

In this era of insanely fast processing computers the memory is often the bottleneck that affects the maximum throughput, Cache is the easiest and cost effective way to achieve high speed memory hierarchy, its performance is very critical for high performance computers. Cache works because programs contain a property called locality of reference, which means they tend to access the same or nearby memory locations again and again over a small period of time, this property of the program is important for cache design and performance. As we know the cache is the closest memory to the CPU this makes it quickly accessible and also very costly according to the memory hierarchy, because of which cache replacement policies play a crucial role in modern operating systems.

Over the years the efficient management of cache memory used to be done by classical strategies such as LRU(Least Recently Used), MRU(Most Recently Used) and FIFO (First-In First-Out) for simple workloads, where the temporary and frequency locality were reliable decider of Future Memory Access.These algorithms provide future references by looking at the past behaviour of programs and follow fixed heuristics to decide which data block should be evicted when the cache is full, while these algorithms work good for predictable access patterns, they often fail to adjust over changing workloads, leading to unavoidable cache misses and performance degradation.This creates a gap for adaptive replacement algorithms that works well with dynamic work loads and regularly adapting to the changing workloads and learning from the past access behaviour. We have used Navies Bayes, Decision Tree and Random Forest ML Algorithms for the comparison between the two methods. By simulating memory access trace we have several features like access time, access frequency, recency rank, etc that are then used to train the models to predict whether a cached item will be reused in the near future or not.

II. RELATED WORK

A. Classical Heuristic Methods

Traditional cache replacement strategies such as LRU, which select the least recently used to evict, while LFU chooses the least frequently used one, have been the pillars of operating systems caches for years because of their easy implementation and low overhead. Although these heuristics are often unable to adapt to changing workload characteristics or to find correlations in access patterns. Many attempted to address some shortcoming by balancing recency and frequency, yet still work on simple rules rather than prediction which makes it static..

B. Machine Learning-Based Approaches

Recent advances have applied machine learning to cache replacement driven by the potential to surpass traditional shortcut policies in performance and adaptability. Deep neural networks have shown promising results allowing them to make more informed cache replacement decisions where access behaviour is highly unpredictable or have long range dependencies.

Reinforcement Learning has also gathered major attention as a sequential decision-making process enables agents to learn optimal policies by communicating with the environment and receiving critique based on cache hits and misses. Another encouraging way is Imitation learning, which aims to copy the performance of conceptually optimal cache replacement strategies by training models to imitate decisions made by the system with future knowledge, for ex:- policies derived through imitation learning have constantly exceeded traditional policies.

Despite these advancements, most ongoing work is dedicated towards deep learning and RL frameworks with less focus on interpretable and resource-efficient models such as navies bayes and decision trees. This opens the question of whether simpler feature-based machine learning models can achieve ambitious performance, particularly in environments such as operating systems, where memory and computational overhead is a critical consideration. However few analyses have systematically evaluated the performance of such feature-based ML policies for cache replacement, especially at large scale,multi-core operating systems. ML models can provide viable quid pro quo between predictive accuracy, adaptability, computational efficiency and convenience in deployment.

This void encourages our work to focus on benchmarking classic machine learning algorithms including Naive Bayes, Decision Trees,Random Forest applied to cache replacement, emphasizing their efficiency in simulated operating system environments. By doing so, we seek to provide new insights potentially offering more practical solutions for real-world adoption.

III. METHODOLOGY

A. *System Overview*

Our cache simulation framework models the behavior of a typical operating system cache . As the system processes a synthetic or real memory access trace, it documents every event such as cache hits, misses, and replacement decisions. Every time a cache eviction is required, instead of relying only on traditional algorithms like LRU,the simulator invokes a trained machine learning model to predict which cache item is least likely to be accessed in the immediate future. This prediction is then used to guide the eviction process, thus integrating ML-driven intelligence into the simulated environment. The system supports interchanging between various ML models and classical policies for fair competitive analysis.

B. *Datasets Details*

Our dataset consists of features corresponding to individual cache items,each characterized by several features:

- Last Access Time: The timestamp when the item was most recently used.
- Access Frequency: Total number of times the item has been accessed while in cache.
- Recency Rank: The item's relative position to others based on recency to use.
- Access Type: Read/Write operations
- Cache Item: Number of Items present in a cache.
- Label: If item will be reused and should be kept in the cache it indicates '1' and If item will not likely be used soon then it is a candidate for eviction from the cache and indicates '0'.

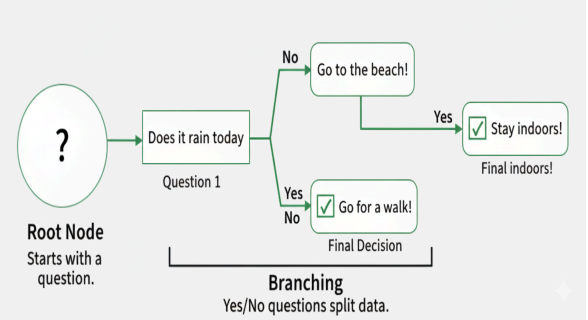
The dataset is generated by running our cache simulation over access traces, regularly capturing features and replacement outcomes for model training and evaluation. This process ensures the dataset produces realistic usage patterns and provides strong groundwork for utilizing feature engineering and normalization to improve model reliability.

C. *Model Explanation*

We have used three algorithms to clarify the usage of cache replacement instead of older methods like LRU, MRU and LFU. The specific models used in cache replacement are discussed below:

- Decision Tree
- Random Forest
- Naive Bayes

1. **Decision Tree** : A Decision Tree is a machine learning algorithm which helps us to make decisions by mapping out different choices and their possible outcomes. It's used in specific tasks like classification and predictions. It's a tree-like structure that starts with one main question i.e known as root node that represents the entire dataset. From root nodes, the tree branches out into different possibilities based on features in the datasets. With the help of Decision Tree, we can possibly see the outcomes by looking at the branches, comparing them and figuring out the best choices.



How Do Decision Trees Work ?

- Start with the Root Node: It begins with a main question, the process begins when a cache miss occurs or the cache is full.
- Ask Yes/No Questions: From the root, the tree asks a yes/no questions to split the data based on attributes.

  For Example: Has this cache block been accessed more than 0 times since it arrived?

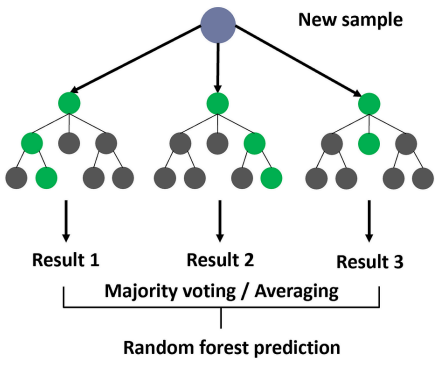
- Branching Based on Answers: Every question leads to different branches:

  If the answer is yes, the cache has low priority.

  If the answer is no, the cache has high priority.

- Continue Splitting: The algorithm asks more deeper questions to differentiate between similar cache blocks.
- Reach the Leaf Node: The process ends at the leaf node and makes their final decision or prediction. Whether the cache block should be replaced according to priority.

2. **Random Forest** : A Random Forest is a machine learning algorithm that uses a combination of many decision trees to make better predictions. In this, every tree is displayed at different random parts of the data and by combining them their results are concluded on the basis of averaging regression or classification voting. This helps in improving accuracy and reducing error as compared to decision trees. It's best to work even if some data is missing so you don't always need to fill in the gaps yourself.



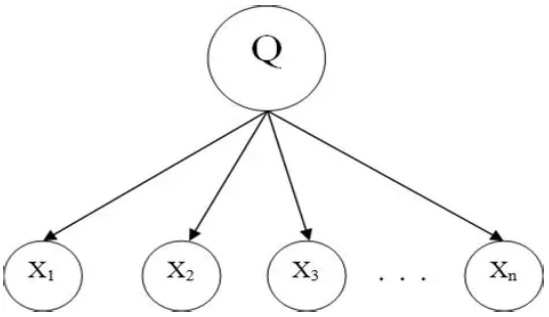
How Do Random Forests Work ?

- Create Many Decision Trees: In this cache hardware creates several independent predictor

units like A predictor that looks at cache time, frequency or type.

- Pick Random Features: When building each tree, it sees a random subset of your features and then helps the trees stay different from each other.
- Each Tree Makes a Prediction: The cache controller asks all trees to vote on specific cache items. Like there are many trees, one may work on counts, another must be worked on recency or time and then make predictions according to it.
- Combine the Predictions: It works on the basis of counting votes for the label. For Example: Vote for Keep-2 and Votes for Evict-1. Then the specific cache item is kept according to voting.

3. **Naive Bayes** : Naive Bayes is a machine learning algorithm that predicts the category of a data point using probability. This algorithm assumes that all features are independent of each other. Its performance is good in many real-world applications. Naive Bayes algorithm works on Bayes' Theorem to classify data on the basis of probability of different classes given the features of the data. It is named as "Naive" because it assumes the presence of one feature of one feature does not affect other features. The "Bayes" part of the name refers to its basis in Bayes' Theorem.



How Do Naive Bayes Work ?

- Terminology: Predict if a cache item should be kept or evicted. Here,

  y(Class Label): Evict(0), Keep(1)

  X(Features Vector): x1,x2,x3,x4

  x1: last_access_time

  x2: access_count

  x3: recency_rank

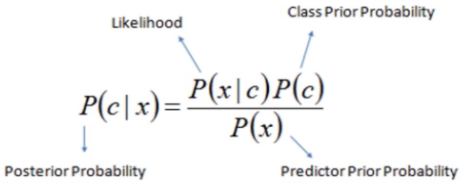
  x4: access_type

  Example : X=(100ms,1,15,Write) y=Evict

- Naive Assumption: The "Naive" assumption pretends that every feature is independent of the others, given the decision.

  P(x1,x2,x3,x4|y)=P(x1|y).P(x2|y).P(x3|y).P(x4|y)

  Thus, Bayes' Theorem:



- Constructing the Naive Bayes Classifier: When the cache is full and needs to pick a victim, it calculates the posterior probability for both options and picks the winner.

  The algorithms computes and shows two scores:

  1. Probability of Evicting
  2. Probability of Keeping

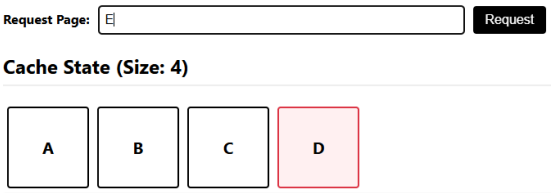
$$\hat{y} = \arg\max_y P(y) \cdot \prod_{i=1}^{n} P(x_i|y)$$

D. Baseline Policies

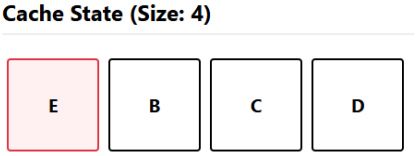
To evaluate the effectiveness of the proposed Random Forest, Decision tree, Naive Bayes based cache replacement policy, it is essential to establish a baseline for comparison. We selected a set of standard deterministic replacement algorithms that are widely implemented in modern hardware. These policies rely on fixed shortcuts—such as temporal locality or access frequency—to make eviction decisions. By comparing our model against these industry standards, we can demonstrate the performance gains achieved by introducing machine learning into the cache hierarchy. The specific baseline policies evaluated are described below:

1. Recency-Based Policies: LRU, MRU
2. Frequency-Based Policies: LFU

- **LRU(Least Recently Used)** : Least Recently Used is a cache replacement algorithm that is used to replace the cache when the space is full. The name of the algorithm suggests that when the cache memory is full, LRU picks the data i.e. least recently used in the state block and removes it in order to make space for new data.The priority of the data in the cache changes on the basis of need of the data.



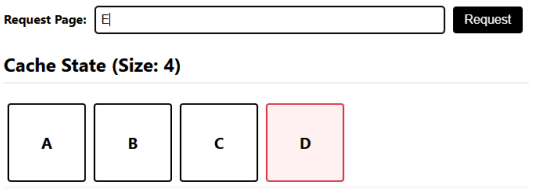
After the E enters the cache state it will replace A as A is the least recently used in comparison of D.



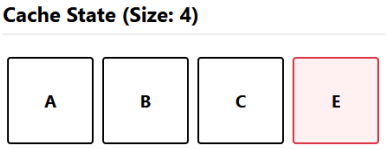
Limitation of LRU: 1. If you have a loop of 5 blocks (A-B-C-D-E) but a cache size of only 4, LRU will evict A right before you need it again. It yields a 0% Hit Rate.

2. If the CPU reads a huge file that is larger than the cache, LRU fills the entire cache with this use-once data, kicking out all the important, frequently used data.

- **MRU(Most Recently Used)** : Most Recently Used is a cache replacement policy i.e. direct opposite of LRU. In this algorithm, instead of evicting the oldest data we evict the most recently accessed data. It operates under the assumption that if the data has just been accessed, then it is a part of the sequence that won't be repeated soon, making it the best candidate to evict.

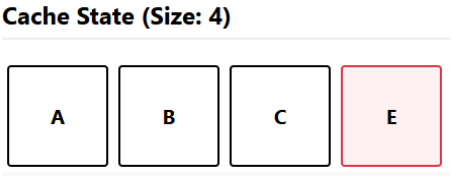


After the E enters the cache state it will replace D as D is the most recently used in the cache state.



Limitation of MRU: MRU is significantly less effective than LRU in case of general, random or mixed workload traffic in modern operating systems.

- **LFU(Least Frequently Used)** : Least Frequently Used is a cache replacement algorithm in which the least frequently accessed cache block is removed when the cache reaches its capacity. In this algorithm, we take into account how much a page is accessed and how recently it was accessed. If a page has higher frequency than another, it cannot be removed or replaced by another cache block.



Like in this If the E has more accessed frequency than another and A,B,C has 1 frequency then the A will be replaced as handled by the FIFO method.

Limitation of LFU: 1. LFU is non-adaptive even if a newly loaded block (count = 1) is immediately the primary candidate for eviction, even if it has just become the center of a new, highly active loop.

2. When multiple blocks have the same low access count then it often falls back to a secondary which is inefficient like the FIFO method.

Policy	LRU	MRU	LFU
Eviction Target	The oldest block (bottom of stack)	The newest block (top of stack)	Lowest access count
Core Principle	Temporal Locality (recent use implies future use)	Cyclic Locality (recent use implies transient use)	Popularity (Used most often is most valuable)
Major Limitation	**Scan Intolerance** (Streaming data pollutes cache)	**General Inefficiency** (Fails standard temporal locality)	**Historical Baggage** (Obsolete high-count data remains protected)

IV. Results

A. *Experimental Setup*

We analyzed our intelligent cache replacement system using various random samples of real world memory access traces, these experiments were conducted in a controlled simulation environment designed to imitate an operating system cache. For each machine learning model (Naive Bayes, Decision Tree, Random Forest), we reviewed baseline comparisons against classical policies such as LRU and LFU.

B. *Evaluation Metrics*

Cache Hit ratio: Portion of memory accesses served by cache.

Miss Ratio: Portion of accesses resulting in cache misses.

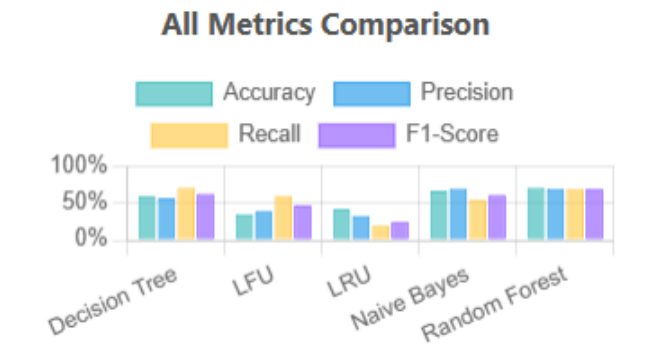
C. *Comparative Analysis*

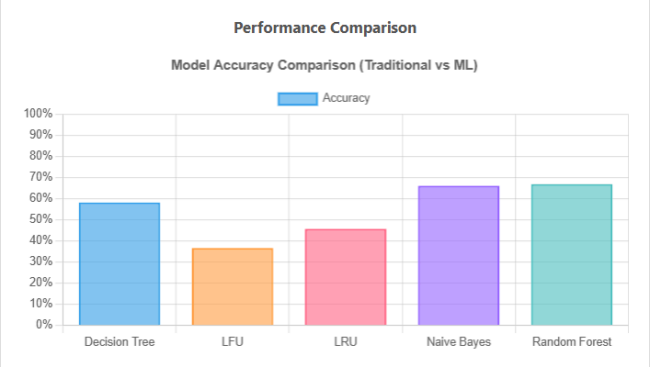
In all examined configurations, our machine learning models outperformed the classical methodologies such as LRU and LFU on hit and miss rates. For illustration, the Random Forest-based policy achieved a 39% higher hit rate relative to LRU under the examination, while maintaining moderate computational overhead. Naive Bayes and Decision Tree models also showed strong performance, on specific workloads.

Performance gains remained steady as cache size increased and for diverse workload patterns, validating the scalability of ML based approaches.

Comparison Results

Model	Accuracy	Precision	Recall	F1-Score	Features Used
Decision Tree	59.40%	56.44%	**70.66%**	62.75%	last_access_time, access_count, recency_rank, access_type
LFU	35.10%	38.90%	59.71%	47.11%	access_count, last_access_time
LRU	41.70%	32.75%	19.42%	24.38%	last_access_time, recency_rank
Naive Bayes	66.40%	**69.79%**	53.93%	60.84%	last_access_time, access_count, recency_rank, access_type, cache_item
Random Forest	**70.50%**	69.48%	69.63%	**69.56%**	last_access_time, access_count, recency_rank, access_type, cache_item





The evaluation measures model performance across four key metrics- accuracy, precision, recall, and F1-score and the features sets used in each approach.
Our results highlight the potential of accountable, lightweight machine learning models for OS-level cache management. ML-based replacement policies consistently capture more subtle usage patterns than fixed policies, leading to concrete performance improvements. While deep learning and RL approaches may offer additional gains, they have greater computational cost. Our findings position naive bayes and random forest as promising middle-ground solutions to high-performing yet practical for real world integration.

While our results are promising, several aspects remain for further development (1) Larger and more diverse datasets:Expanding experiments to border workload representations will enhance model relevance. (2) Hardware implementation: Exploring the potential of deploying a ML-based cache replacement in actual operating system environments or hardware cache controllers, focusing on latency.(3) Continuous learning: Adapting models to learn constantly from evolving access patterns can further boost versatility.

Conclusion

In this study, we investigated intelligent cache replacement strategies using machine learning algorithms, specifically Naive Bayes, Decision Tree, and Random Forest, and compared their effectiveness against classical policies such as LRU and LFU. By simulating cache behaviour and utilizing a rich feature set derived from memory access patterns, our models demonstrated marked improvements in cache hit ratio, predictive accuracy, and overall efficiency.

Among the models tested, Random Forest consistently outperformed traditional policies in both accuracy (70.50%) and F1-score (69.56), while Decision Tree thrived in recall and Naive Bayes in precision. These outcomes highlight the practical benefits of feature-driven machine learning for cache management, offering a balance between interpretability, adaptability, and computational efficiency.

References

P. R. Pratheeksha and R. Kavan, “Machine Learning-Based Cache Replacement Policies: A Survey,” *International Journal of Engineering and Advanced Technology*, vol. 10, no. 6, pp. 81-87, 2021. [Online].

Y. Chen, B. Zhang, and L. Chen, “Hadoop-Oriented SVM-LRU (H-SVM-LRU): Intelligent Cache Replacement Algorithm,” *arXiv preprint arXiv:2309.16471*, 2023. [Online].

T. Liu, K. Lee, A. Garg, and S. Shakkottai, “An Imitation Learning Approach for Cache Replacement,” in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 6309–6318. [Online].

Random Forest Algorithm in Machine Learning," *GeeksforGeeks*, Feb. 21, 2024. [Online].

X. Lin, J. Li, and D. A. Wood, “Applying Deep Learning to the Cache Replacement Problem,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 243-254. [Online].

H. Rodriguez, T. Young, Q. Chen, et al., “Learning Cache Replacement with Cacheus,” in *Proceedings of the 19th USENIX Conference on File and Storage Technologies*, 2021, pp. 238-253. [Online].

Naive Bayes Classifier in Machine Learning," *GeeksforGeeks*, Apr. 16, 2024. [Online].

S. M. Tabassum, S. S. Mishra, and R. K. Mishra, “Priority Cache Object Replacement by Using LRU, LFU and Their Combinations,” *Transstellar Journal*, vol. 6, no. 1, pp. 23–29, 2020. [Online].

K. Nasr, V. Chidambaram, and I. Abraham, “Performance comparison of cache replacement algorithms,” *Semanticscholar*, 2019. [Online].

L. Zhang and G. Wu, “A new cache replacement scheme based on backpropagation neural networks,” in *Proceedings of the 1997 ACM Symposium on Applied Computing*, 1997, pp. 552–556. [Online].

E. Bosch, “Measurement-based Modeling of the Cache Replacement Process,” in *Proceedings of the 19th Real-Time and Embedded Technology and Applications Symposium*, 2013, pp. 131–139. [Online].

S. Liu, R. Zhang, and Y. Wang, “Advancements in cache management: a review of machine learning techniques,” *Frontiers in Artificial Intelligence*, vol. 8, pp. 1–15, Feb. 2025. [Online].